Experiment 2- Stack Operations using an Array

Learning Objective: Student should be able to perform operations on Stack data structures.

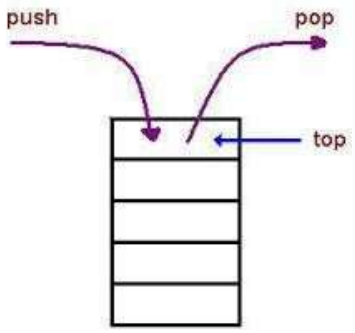
Tools: C under Windows or Linux environment.

Theory: Develop a code for stack using an array (Menu driven program)

Stack is a data structure which works in last in first out manner i.e. LIFO manner. The data which comes first, goes at last and which comes at last goes first.“

Or

“Stack is a collection of data, whose elements are added and removed at one end called top.”



Functions that can be performed on the Stacks using an ADT are as follows:

1. **To push element push (Stack s, Element e):** In which element e has to be pushed instack s, and top position will be updated.

2. **To pop element Element e =pop(Stack s):** In which element e has to be popped fromstack s, and top position will be updated.

3. **To check whether stack is empty, function is isEmpty(Stack s):** which will returnBoolean value, if stack is empty true value will be returned otherwise false value will be returned.

4. **To check whether stack is full, function is isFull(Stack s):** which will return Booleanvalue, if stack is full true value will be returned otherwise false value will be returned.

5. **To find top position, function top(Stack s):** which will return position of top i.e.integer value.

ALGORITHMS:

PUSH

Step 1: If TOP >= SIZE – 1
then Write "Stack is Overflow"
Step 2: TOP = TOP + 1
Step 3: STACK [TOP] = X

POP

Step 1: If TOP = -1 then
Write "Stack is Underflow" **Step 2:** Return STACK [TOP] **Step 3:** TOP = TOP - 1

PEEK

Step 1: If TOP=-1 then
Write "Stack is Underflow"
Step 2: Return STACK [TOP]

ADVANTAGE:

1. Easy to understand and implement using an array.

DISADVANTAGES:

1. Wastage of memory space in stack data structure as reutilization of the space is not possible in it.
2. Only one end is open for all the operations to be performed.

APPLICATIONS:

1. Converting the infix expression into its equivalent prefix and postfix forms and evaluate them.
2. Reversing the contents of the string.
3. Converting the decimal number into its equivalent binary number.
4. Checking the parentheses for the program.

Learning Outcome: The student should have the ability to LO1: understand and implement stack data structures.
LO2: perform operations like push, pop, peek and traversing on stack data structures.

Course Outcomes: Upon completion of the course students will be able to apply operations like push, pop, peek, search and traverse on Stack data structure.

Code:

```
#include<stdio.h>
int top=-1,x,c;
int MaxSize=100;
int stack[100];
int isEmpty()
{
   if(top==-1)
      return 1;
   else
      return 0;
}
int isFull()
{
   if(top==MaxSize-1)
      return 1;
   else
      return 0;
}
void push()
{
    if(isFull())
      printf("Stack Overflow\n");
    else
    {
       top++;
       stack[top]=x;
    }
}
void pop()
{
    if(isEmpty())
      printf("Stack Underflow\n");
    else
    {
       x=stack[top];
       top--;
       printf("Element popped is %d\n",x);
    }
}
void peek()
{
    if(isEmpty())
      printf("Stack Underflow\n");
    else
    {
       x=stack[top];
       printf("Element at top is %d\n",x);
```

```
    }
}
void display()
{
    if(isEmpty())
      printf("Stack Underflow\n");
    else
    {
       for(int i=top;i>=0;i--)
       {
            printf("| %d |\n",stack[i]);
            printf("-----\n");
       }
    }
}
int main()
{
   while(c!=5)
   {
          printf("Enter Choice(1.Push 2.Pop 3.Peek 4.Display 5.Exit): ");
          scanf("%d",&c);
          switch(c)
          {
                case 1:
                   printf("Enter element to push: ");
                   scanf("%d",&x);
                   push();
                   break;
                case 2:
                   pop();
                   break;
                case 3:
                   peek();
                   break;
                case 4:
                   display();
                   break;
                case 5:
                   break;
                default:
                   printf("Invalid choice\n");

          }
   }
}
```

Output:

```
Enter Choice(1.Push 2.Pop 3.Peek 4.Display 5.Exit): 1
Enter element to push: 3
Enter Choice(1.Push 2.Pop 3.Peek 4.Display 5.Exit): 1
Enter element to push: 4
Enter Choice(1.Push 2.Pop 3.Peek 4.Display 5.Exit): 4
| 4 |
-----
| 3 |
-----
Enter Choice(1.Push 2.Pop 3.Peek 4.Display 5.Exit): 2
Element popped is 4
Enter Choice(1.Push 2.Pop 3.Peek 4.Display 5.Exit): 4
| 3 |
-----
Enter Choice(1.Push 2.Pop 3.Peek 4.Display 5.Exit): 3
Element at top is 3
Enter Choice(1.Push 2.Pop 3.Peek 4.Display 5.Exit): 5
```

Conclusion:

For Faculty Use

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				